

JENKINS

1. Explain about your CI/CD project workflow??

Yeah sure! **First we have a checkout stage**, it starting from github once the developer push the code it triggers the jobs using webhook mechanism, **The second stage we have a build stage**, Here we using maven for build operation to convert the code into jar or war file, all the build information we configure in pom.xml file(project object model) as a devops engineer i integrate the maven and jenkins. **The third stage is the testing stage**, where we use a sonarqube scanner for code quality analysis and In code we have a threshold value that should meet the threshold value then only it is a success code or it fails. **Then fourth is the docker stage**, The dockerfile converts to docker image and we have a **trivy to scan the docker image**. **The fifth stage is the Artifact stage**, we use nexus (private repository). We use Nexus in Jenkins CI/CD pipelines as a central artifact repository to store, version, and manage build outputs (like `.jar` or Docker images`), ensuring consistent deployments, rollback capability, and secure dependency management. **The sixth stage is container storage stage** ECR (elastic container registry) or docker hub is used to store the containers and **finally we have production stage**, production is success we get the via post block or if it fails it moves to rollback . based on the post block notification (success and failure).

2. Jenkins pipeline code?

There is an two types of pipeline codes are there

- scripted and Declarative.

3. Write some sample CI/CD pipeline ?

Yeah sure !

```
pipeline {  
    agent any  
  
    environment {  
        DOCKER_IMAGE =  
        REGISTRY =  
    }  
  
    stages {  
        stage ('checkout') {  
            steps {  
                git branch :  
                url :  
            }  
        }  
    }  
  
    stage ('Test') {  
        steps {  
            sh "mvn test"  
        }  
    }  
  
    stage ('Docker Build') {  
        steps {  
            sh "docker build -t $"  
        }  
    }  
  
    stage ('Push to Registry') {  
        steps {
```

```

        withcredentials([usernamepassword(credentialsID: ,
usernamevariable: ,passwordVariable: )) {
            sh " "
            sh "docker push "
        }
    }
}

```

```

stage ('Deploy') {
    steps {
        script {
            try {
                sh "kubectl set image deployment/myapp "
                sh "kubectl rollout status deployment/myapp "
            }
            catch (exception e) {
                error (" ")
            }
        }
    }
}

```

```

post {
    success {
        echo
    }
    failure {
        script {
            sh " "
        }
    }
}

```

```
always {  
    echo  
    cleanWS()  
}  
}  
}
```

4 . Explain about your master and slave setup in Jenkins ?

In our Jenkins setup, We have one master node and 4 slave nodes. The master node manages job scheduling and configuration, while slave nodes (agents) execute builds on distributed machines. This ensures scalability, load balancing, and supports multiple environments for CI/CD pipelines.

5. How to add a secret in Jenkins ?

In Jenkins, we securely add secrets under **Manage Jenkins** → **Credentials**, where they're encrypted and stored safely. We then access them inside pipelines using the `withCredentials` block with the secret's credential ID. using the environment variable & jenkins credential plugins , for external we can use ASM.

6. My pipeline is taking too much time to build how to manage this ?

If my Jenkins pipeline is slow, I first analyze which stages consume the most time using stage view. Then I apply optimizations like **parallel stages, caching dependencies, using conditional builds, optimizing Docker layers, and distributing load to agents** — these improvements usually reduce build time significantly.

7. How will I take a backup in Jenkins ?

To back up Jenkins, I take a backup of the `$JENKINS_HOME` directory — it contains all jobs, configurations, plugins, and credentials. I usually automate it using the ThinBackup plugin or custom scripts to store compressed backups on cloud storage like S3. During restoration, I simply replace the `$JENKINS_HOME` folder.

8. What are plugins in Jenkins ?

Plugins in Jenkins are extensions that add extra functionality or integrations — for example, Git plugin for SCM, Docker plugin for container builds, or Slack plugin for notifications. Jenkins uses over 1,800 community plugins, making it highly flexible and customizable for any CI/CD setup.

9. explain the Jenkins shared library ?

A Jenkins Shared Library is a centralized repository of reusable pipeline code that can be imported across multiple Jenkinsfiles. It helps maintain consistency, reduces duplication, and simplifies CI/CD management by allowing common logic (like build, test, deploy steps) to be maintained in one place.

10. How to integrate your CD stages in Jenkins?

I integrate CD stages in Jenkins by adding deployment stages after build and test in the pipeline. The Jenkinsfile handles environment-specific deployments using tools like Docker, Kubernetes, or Ansible. I use approvals for staging/production, Jenkins credentials for secure access, and automated rollbacks for reliability.

11. What is continuous delivery and deployment ?

Continuous Delivery ensures that code is always in a deployable state — deployments to production happen after manual approval. Continuous Deployment goes a step further and automatically deploys every tested change to production without human intervention. Both aim to speed up releases and improve reliability.

12 . Explain the parallel execution ?

Parallel execution in Jenkins allows multiple stages or tasks to run at the same time instead of sequentially. This helps reduce pipeline duration and better utilize resources. It's commonly used for running tests, builds, or deployments across environments simultaneously using the `parallel` directive.

13 . How will you handle the error handling in the Jenkins pipeline?

Error handling in a Jenkins pipeline ensures that failures are caught and managed gracefully without breaking the entire CI/CD process unexpectedly. It can be implemented using `try-catch` blocks in scripted pipelines, the `post` section in declarative pipelines for success, failure, or cleanup actions, and the `catchError` step to handle non-critical errors. Proper error handling also allows sending notifications, logging issues clearly, and marking builds as failed or unstable based on severity, which improves pipeline reliability and maintainability.

14. How will I set up the access between multiple users in Jenkins?

Access between multiple users in Jenkins is managed through Role-Based Access Control (RBAC) and built-in security settings.

By enabling Jenkins security, you can assign users different roles like admin, developer, or viewer, controlling their permissions on jobs, pipelines, and system configurations. This ensures proper segregation of duties, prevents unauthorized changes, and allows collaborative work while maintaining pipeline security.

15. Explain the triggers method in Jenkins?

Triggers in Jenkins are used to automatically start pipelines or jobs based on specific events, eliminating the need for manual execution. Common triggers include SCM polling for code changes, cron-based schedules for periodic builds, webhooks from GitHub/GitLab, and upstream/downstream job triggers. They help automate CI/CD workflows, ensure timely builds and tests, and improve overall development efficiency.

16. One of the developers wrongly committed a sensitive secret informations in GitHub how will you manage this issue?

If a developer accidentally commits sensitive information to GitHub, the first step is to remove the secret from the repository history using tools like `git filter-branch` or the BFG Repo-Cleaner.

Next, immediately rotate or revoke the exposed secrets to prevent any unauthorized access. Finally, update the team's workflow by using environment variables, secret managers, or Jenkins credentials to avoid committing sensitive data in the future and enforce proper code review practices.